INTERNAL · SURMOUNT ENGINEERING

Partner SSO

How a Blossom member becomes a signed-in Surmount member without typing anything — the terms, the flow, the code, the failure modes, and what is still missing.

PROTOCOL
OpenID Connect · Authorization Code + PKCE

OUR ROLE
Relying Party (confidential client)

LIBRARY
authlib · `integrations.django_client`

OUR CODE
408 lines · 4 service modules

TESTS
48 client-side · 33 on the provider mock

BRANCH
`feat/blossom-partner-sso`

Contents

01 Architecture

OpenID Connect · Authorization Code flow · PKCE (S256) · confidential client · just-in-time provisioning.

DECISION	WHY, AND NOT THE ALTERNATIVE
`OpenID Connect`	An open standard. The partner's stack is irrelevant; the contract is identical whether they run Python, Node, Java or Go
`Authorization Code`	Implicit and hybrid put tokens in the URL — browser history, `Referer` headers, proxy logs
`+ PKCE S256`	A stolen code is useless without the verifier. `plain` is the verifier in clear text, so not a challenge at all. RFC 9700 requires PKCE for *all* clients, not just public ones
`Confidential client`	We have a server, so the client secret never touches a browser
`Back-channel exchange`	Identity moves server to server. The browser only ever carries opaque handles
`Link on sub`	Emails change and get recycled. Matching one would hand over the wrong brokerage account
`Opaque session code`	Our auth is JWT and the app is a different origin. A JWT in a redirect URL is a live credential in every log
`Session-issuing, not token-relay`	We mint our own session rather than validating Blossom's token per request. Keeps us decoupled from their uptime and gives every partner one auth model — see §14

THE ORGANISING INVARIANT

There are **two channels**, and which one a value travels on decides how much it is trusted.

- **Front channel** — the browser, via redirects. Readable and forgeable by the user. Carries only random, single-use, short-lived handles.
- **Back channel** — our server calling theirs directly. Inside TLS, secret-authenticated. Carries the identity itself, signed.

Anything an attacker can put in a URL, we check against something they cannot reach. `state` against a server-side record keyed by an `HttpOnly` cookie; `nonce` against a claim inside a signed token; the authorization code against a verifier that never left our server. Ask this of any future change and you will usually get the right answer.

02 The four parts

Blossom FE

The bank's web app. Holds one link and nothing else.

`:5300 · partner`

Blossom BE

The bank's API **and** its OpenID Provider. Owns the login, the password, the identity.

`:9000 · partner`

Whitelabel

The Blossom-branded investing app (`blossom-fe`). No sign-in of its own.

`:4001 · ours`

Surmount BE

Accounts, KYC, custodians, trades. The member never sees this name.

`:8000 · ours`

WHO TALKS TO WHOM

- **Blossom FE → Surmount BE** — one link. A browser navigation, never an API call.
- **Surmount BE ↔ Blossom BE** — the back channel. Where identity actually moves.
- **Whitelabel → Surmount BE** — ordinary API calls, once signed in.
- **Blossom FE ✕ Whitelabel** — never talk to each other.

> #### LOCAL DEVELOPMENT
>
> All four run on `localhost` . Because cookies are scoped by host and **ignore the port**, the two Django apps would otherwise share one `sessionid` and silently overwrite each other's flow state. Both set a distinct `SESSION_COOKIE_NAME` (`surmount_session` / `blossom_session`) to prevent this. It costs nothing in production and saves an afternoon locally.

03 Every term

Everything you will meet in the code or the logs. Read this once and the rest is obvious.

Configuration — set once, never changes

`issuer` FROM BLOSSOM

The one URL Blossom gives us. Everything else about their provider is read from it.

ISSUED BY	Blossom BE
USED BY	Surmount BE
LIVES	Forever
CHECKED	As `iss` on every ID token, exact match

Without it: six URLs by hand, and they could never move them.

`discovery document` FROM BLOSSOM

JSON at a spec-fixed path listing every endpoint, key location and supported algorithm.

ISSUED BY	Blossom BE
USED BY	authlib, at first use
LIVES	Cached 1 hour
PATH	`/.well-known/openid-configuration`

Without it: every endpoint change on their side needs a coordinated deploy on ours.

`client_id` FROM BLOSSOM

Our public name at their provider. Identifies which application is asking.

ISSUED BY	Blossom, at registration
USED BY	The `/authorize` redirect
LIVES	Forever
SECRET?	No — it is in the URL

Also: checked as `aud` on every ID token, so a token minted for another of their clients is refused.

`client_secret` FROM BLOSSOM

Our password at their token endpoint. Proves the back-channel caller is us.

ISSUED BY	Blossom, printed once
USED BY	Surmount BE, HTTP Basic
LIVES	Until rotated
SECRET?	**Yes** — server-side only

Without it: anyone who intercepted a code could redeem it.

`redirect_uri` OURS, THEY ALLOW-LIST IT

Where Blossom may send the browser back to. Matched **exactly**.

OWNED BY	Surmount
REGISTERED AT	Blossom
SENT ON	`/authorize` *and* `/token`
LIVES	Forever

Without exact matching: an attacker appends their own host and receives the code. The most exploited bug in OAuth.

`scope` WE ASK

What we ask to learn. Decides which claims come back.

ASKED BY	Surmount BE
GRANTED BY	Blossom
LIVES	Per request
OURS	`openid profile email`

Ask for less: you get a `sub` and nothing else — not enough to create an account.

The three random values

They look identical. They do three different jobs and each stops a different attack.

`state` ANTI-CSRF

Random string proving this callback answers a request **we** made, **in this browser**.

ISSUED BY	authlib
STORED	Redis, keyed by our session cookie
TRAVELS	Front channel, both ways
LIVES	1 hour · single use

Without it: an attacker sends you a callback carrying *their* code. You end up signed into *their* account and type real details into it.

`nonce` ANTI-REPLAY

Random string Blossom must embed **inside** the signed ID token, proving it was minted for this request.

ISSUED BY	authlib
STORED	Redis, alongside `state`
TRAVELS	Out in the URL, back inside the token
LIVES	1 hour · single use

Without it: a token captured from a sign-in last month replays today and still verifies.

`code_verifier / challenge`

PKCE · ANTI-THEFT

We invent a secret, send only its **SHA-256 hash** up front, reveal the secret when collecting the token.

ISSUED BY	authlib
VERIFIER	**Never leaves our server**
CHALLENGE	The hash — in the URL
LIVES	1 hour · single use

Without it: anyone who intercepts the code — extension, shared machine, leaky mobile URL handler — can redeem it.

`front vs back channel` THE CORE IDEA

Not a value — a distinction. **Front** = through the browser. **Back** = our server calling theirs.

FRONT CARRIES	Opaque handles only
BACK CARRIES	The identity, signed
FRONT IS	Forgeable by the user
BACK IS	TLS + secret-authenticated

If identity travelled on the front channel: the member could edit it, and so could anything on their machine.

What moves during the hand-off

`authorization code` FROM BLOSSOM

A receipt saying "this person authenticated here". Worthless on its own.

ISSUED BY	Blossom BE
REDEEMED AT	Their `/oauth/token`
TRAVELS	Front channel, one hop
LIVES	**60 s · single use**

Why so short: it passes through the browser, so it will end up in a log. A minute later it is worth nothing.

`id_token` THE ACTUAL IDENTITY

A **signed JWT** stating who this person is. The whole point of the exercise.

ISSUED BY	Blossom, their private key
VERIFIED BY	authlib, against their JWKS
TRAVELS	**Back channel only**
LIVES	1 hour · used in a second

Why signed: a signature is verifiable on its own. We hold only the public key and could not forge one.

`sub` THE PERMANENT LINK

Blossom's forever-identifier for one human. The most important value in the system.

ISSUED BY	Blossom (their user UUID)
STORED IN	`api_partnerssoidentity`
LIVES	**Forever.** Never changes, never reused
MATCHED ON	Every hand-off

If it were an email: emails get changed and recycled — a recycled one would hand a stranger someone else's brokerage account.

`access_token (Blossom's)` NARROW ON PURPOSE

Permission to read `/userinfo` once. Not a session on Blossom; cannot move money.

ISSUED BY	Blossom BE
USED BY	Surmount BE, at most once
TRAVELS	Back channel only
LIVES	**5 minutes** · no refresh

Worst case if leaked: it discloses the profile of the member already being handed over. Nothing more.

`JWKS + kid` THE PUBLIC KEYS

Blossom publishes the **public half** of their signing key. `kid` names which key signed a token.

PUBLISHED BY	Blossom BE
FETCHED BY	authlib, cached 1 hour
LIVES	Until rotated
ROTATION	Unknown `kid` → refetch once

Without kid: rotating a signing key needs a coordinated downtime window with every client.

`session_code` THE LAST HOP

An opaque handle we swap for our own JWT, so the token never appears in a URL.

ISSUED BY	Surmount BE
REDEEMED BY	Whitelabel, over POST
TRAVELS	The final redirect URL
LIVES	**60 s · single use**

Without it: our JWT lands in history, `Referer` headers and access logs — live, for its full lifetime.

Sessions

`Blossom session cookie` THEIR LOGIN

Proves the member is signed in to the bank. What makes `/oauth/authorize` invisible.

ISSUED BY	Blossom BE
HELD BY	The browser
READ AT	`/oauth/authorize`
LIVES	2 weeks (their rule)

Without it: the member sees **Blossom's own** login page — never ours. That is what makes this SSO.

`Surmount access JWT` OUR SESSION

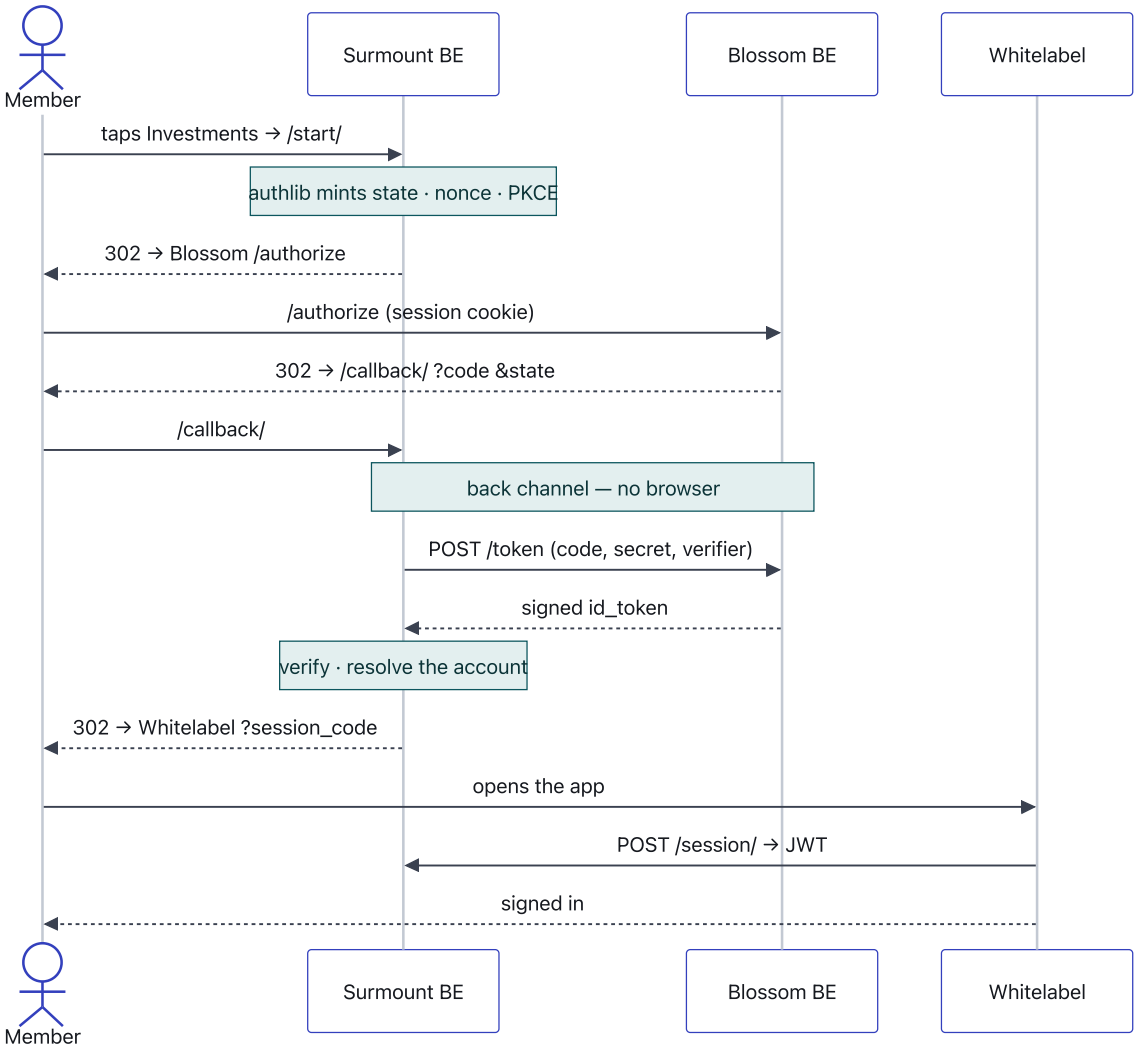
Our own token. Sent on every Whitelabel API call after the hand-off.

ISSUED BY	Surmount BE
HELD BY	Whitelabel, localStorage
LIVES	**60 days** (local: ~2 years)
REFRESH	30 days

Note: this is **ours, not Blossom's**, and it outlives everything they issued. See §12.

> THE ONE THAT SURPRISES PEOPLE
>
> **Blossom grants us almost nothing.** Every credential they issue is spent within a second of the hand-off. The 60-day session is **ours**, minted by us — which is why revocation is our problem, not theirs.

04 The flow



Two visible redirects. Under a second, start to finish.

On the wire

1 · THE LINK, ON BLOSSOM'S DASHBOARD

FRONT CHANNEL · BLOSSOM'S HTML

```
<a href="https://api.surmount.ai/api/sso/blossom/start/">Investments</a>
```

A plain navigation. Blossom's frontend makes no API call and holds nothing.

2 · WE BEGIN THE FLOW

FRONT CHANNEL · SURMOUNT RESPONDS

```
GET /api/sso/blossom/start/

302 Found
Location: https://id.blossom.com/oauth/authorize
    ?response_type=code
    &client_id=surmount-investing
    &redirect_uri=https%3A%2F%2Fapi.surmount.ai%2Fapi%2Fsso%2Fblossom%2Fcallback%2F
    &scope=openid+profile+email
    &state=s-hp-ADERCXb5BgBiGD6gsJB8AulaiY-K2noMKOSVG8
    &nonce=yAeeq9y_RGebevliVxEETiu7f-MCm97AUY7rzxCVNN8
    &code_challenge=ztuG_XUL3i8OuL-p__LTVU9Xs_TOIydXlAqLPT8SQLA
    &code_challenge_method=S256
Set-Cookie: surmount_session=…; HttpOnly; SameSite=Lax; Secure
```

- `state`, `nonce` and the PKCE **verifier** are stored in Redis.
- The session cookie carries only an expiry marker — authlib keys the real data by `state`.
- The URL carries the first two and only the **SHA-256 hash** of the third.

3 · BLOSSOM AUTHORIZES

FRONT CHANNEL · BLOSSOM RESPONDS

```
GET /oauth/authorize?…
Cookie: blossom_session=<their own session>

302 Found
Location: https://api.surmount.ai/api/sso/blossom/callback/
    ?code=5c0L7HDO6dWZlGwvwhcClJZFHCgGkxRCs5KmDJAlVv
    &state=s-hp-ADERCXb5BgBiGD6gsJB8AulaiY-K2noMKOSVG8
```

Invisible to the member. If they were *not* signed in, they would see **Blossom's own** login page — never ours. That is what makes this SSO rather than a second account.

4 · STATE CHECK, THEN THE BACK CHANNEL

BACK CHANNEL · SERVER TO SERVER

```
POST https://id.blossom.com/oauth/token
Authorization: Basic <base64 client_id:client_secret>
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&code=5c0L7HDO6dWZlGwvwhcClJZFHCgGkxRCs5KmDJAlVv
&redirect_uri=https%3A%2F%2Fapi.surmount.ai%2F…%2Fcallback%2F
&code_verifier=<the value that never left our server>

→ 200 OK
{ "access_token": "vLQ1yZ0…", "token_type": "Bearer",
  "expires_in": 300, "id_token": "eyJhbGciOiJSUzI1NiIsImtpZCI6…" }
```

Three independent proofs must line up: the code, our client secret, and the PKCE verifier. Stealing any one is not enough.

5 · THE ID TOKEN, AND THE SIX CHECKS

WHAT BLOSSOM SIGNED

```
{
  "iss": "https://id.blossom.com",        ← must match, exactly
  "sub": "8c14b7e2-0000-4000-8000-9d3e…", ← the permanent link
  "aud": ["surmount-investing"],          ← minted for us
  "exp": 1786029000,
  "iat": 1786025400,
  "auth_time": 1786025380,
  "nonce": "yAeeq9y_RGebevliVxEE…",       ← the one we generated
  "given_name": "Nadia", "family_name": "Halim",
  "email": "nadia@example.com", "email_verified": true,
  "picture": "https://id.blossom.com/avatar/8c14b7e2.svg"
}
```

CHECK	WHAT IT STOPS
signature, against JWKS	A forged token
iss exact match	A token from a different provider we also trust
aud = our client id	A token minted for another of Blossom's clients
exp / iat, 60 s skew	A token lifted from an old session
nonce = what we generated	A replay of an earlier sign-in
alg allow-list	`alg: none`, and HS256-with-the-public-key confusion

All six are performed by `authlib`'s `parse_id_token`. Drop any one and something specific becomes possible.

6 · BACK TO THE WHITELABEL, THEN TRADE THE HANDLE

FRONT CHANNEL, THEN A POST

```
302 Found
Location: https://invest.blossom.com/sso/callback?session_code=vA2o7nkNkWL4…

_____________________________________________

POST /api/sso/session/
Content-Type: application/json

{ "session_code": "vA2o7nkNkWL4…" }

→ 200 OK
{ "access": "eyJ0eXAiOiJKV1Qi…", "refresh": "eyJ0eXAiOiJKV1Qi…" }
```

> **WHY THE JWT IS NOT SIMPLY IN THE REDIRECT**
>
> URLs are not private. They land in browser history, in the `Referer` header of the next outbound request, in proxy and access logs, and in whatever the member pastes into a chat window. A JWT in a URL is a live credential in all of those for its full lifetime. A 60-second single-use handle is worth nothing by the time anyone reads it out of a log.

7 · EVERYTHING AFTER THIS

The Whitelabel **never decodes the token** — it calls `/api/user/detail/` . A token is a *claim*, and only the backend can verify it. If the frontend trusted its contents, editing localStorage would let anyone claim to be anyone. Blossom is out of the loop entirely from here.

05 Which account they get

`services/sso/linking.py` — three branches, every one ending with a signed-in member.

RESOLVE_USER()

```
1. PartnerSSOIdentity(partner_slug="blossom", subject=sub)
     found    → that user
     closed   → stop, ?error=account_closed

2. User(email=…, organization=<partner org>)
     found    → adopt it, write the link
     closed   → stop, ?error=account_closed

3. neither  → create:
       CoreUser.get_or_create(email=…)   reused if they already have one
       User(organization=…, password=<random>, origin="sso:blossom")
       PartnerSSOIdentity(...)           so branch 3 never runs again
```

- **Branch 1** — one indexed query on `(partner_slug, subject)`. No decision, no writes.
- **Branch 2** — someone who signed up on the Whitelabel before the integration existed. Their portfolio, KYC record and history carry across. **See §12 — this branch needs a guard.**
- **Branch 3** — routed through `SignupAPIView`, not `create_user`, so the org's flags actually take effect: `skip_email_verification`, cross-tenant KYC linkage, banners, activity tracking.

WHY NO PHONE, NO FORM

The partner already told us who they are; a signup form would only ask them to retype it. Everything past identity — phone, address, citizenship, SSN — belongs to KYC, which collects it properly and verifies the phone with an SMS OTP.

THE PASSWORD

`secrets.token_urlsafe(32)`, unusable by design. There is no password for an attacker to guess and none for the member to lose — which is also why **every dead end must return them to Blossom**, never to a sign-in form they could not pass.

06 Code map

FILE	LINES	JOB
`services/sso/providers.py`	131	Registers one authlib OIDC client per partner from config. The registry is built once per process
`services/sso/identity.py`	139	`Identity` dataclass, the five error types, and `from_claims()` — the seam
`services/sso/linking.py`	242	Find, adopt or create the Surmount account. **Has never heard of OIDC**
`services/sso/session.py`	50	Stash the JWT behind a 60-second single-use handle
`views/sso_views.py`	239	The three endpoints, and the error mapping
`routes/sso_urls.py`	19	Three routes, partner as a path segment
`models/partner_sso.py`	61	The link table

408 lines of our code. `authlib` is imported in exactly two files.

WHAT AUTHLIB DOES, SO WE DO NOT

- ✓ Builds the authorization URL
- ✓ Generates, stores and validates `state`, `nonce` and the PKCE verifier
- ✓ Exchanges the authorization code
- ✓ Fetches and caches the discovery document and the JWKS
- ✓ Resolves the signing key by `kid`, and refetches once on rotation
- ✓ Validates the ID token — all six checks

THE SEAM

Above `Identity` is protocol — redirects, signatures, secrets, whatever a partner's platform speaks. Below it is Surmount — organizations, core users, KYC. `Identity` is the only type that crosses, and it is only ever constructed **after every check has passed**, so the trust boundary is a type boundary rather than a comment.

THE THREE ENDPOINTS

METHOD + PATH	AUTH	DOES
`GET /api/sso/<partner>/start/`	none	Mints state · nonce · PKCE, redirects to the partner
`GET /api/sso/<partner>/callback/`	none	Verifies, resolves the account, redirects to the Whitelabel
`POST /api/sso/session/`	none	Trades the handle for `{access, refresh}`

All three are unauthenticated by necessity and share the `sso` throttle scope (30/min), so they cannot exhaust the general anonymous allowance.

> **WHY THE PARTNER IS IN THE URL AND NOT A SETTING**
>
> One deployment serves any number of partners, and no stale environment variable can hand a member to the wrong one. The URL segment **is** the key into `SSO_PARTNERS`. An unknown slug redirects with `?error=sso_unavailable`.
>
> The path itself is not in any spec — OIDC only defines the *provider's* endpoints. But `<provider>` in the path is the mainstream convention: Spring Security uses `/oauth2/authorization/{id}`, django-allauth `/accounts/<provider>/login/`, NextAuth `/api/auth/signin/<provider>`.

07 The data model

API_PARTNERSSOIDENTITY — THE ONLY NEW TABLE

```
partner_slug    varchar(64)    "blossom"
subject         varchar(255)   the partner's sub — immutable, never reused
user_id         uuid FK        → api_user
core_user_id    uuid FK        → api_coreuser

from BaseModel:
id, created_at, modified_at, deleted_at,
created_by, modified_by, deleted_by, is_deleted

UNIQUE (partner_slug, subject)
INDEX  (partner_slug, subject)
```

- **The unique constraint is the design.** It makes visit #2 a single lookup, and stops two simultaneous first arrivals creating two accounts for one person.
- **No other schema changed.** Migration `0306_partnerssoidentity_and_more`.
- `core_user` is denormalised from `user.core_user` so a link can be resolved without a join through `User`.

> **WHAT THE CONSTRAINT DOES NOT STOP**
>
> `UNIQUE (partner_slug, subject)` prevents one subject mapping to two users. It does **not** prevent two subjects mapping to the *same* user — which is reachable through the adopt branch. See §12, item 1.

08 Configuration reference

BACKEND/.ENV

```
# Which partner, and where their members belong
SSO_PARTNER_SLUG=blossom
SSO_PARTNER_DISPLAY_NAME=Blossom
SSO_PARTNER_ORG_ID=<tenant uuid — NOT the parent Surmount org>
SSO_LANDING_URL=https://invest.blossom.com/sso/callback
SSO_SESSION_CODE_TTL_SEC=60

# The only URL the partner gives us. Must be https outside DEBUG.
SSO_OIDC_ISSUER=https://id.blossom.com
SSO_OIDC_CLIENT_ID=surmount-investing
SSO_OIDC_CLIENT_SECRET=<from the partner, via a secrets manager>
SSO_OIDC_REDIRECT_URI=https://api.surmount.ai/api/sso/blossom/callback/
SSO_OIDC_SCOPES=openid profile email

THROTTLE_SSO_RATE=30/min

# More than one partner? Set this to the whole map instead
# and leave the discrete variables unset.
# SSO_PARTNERS_JSON={"blossom": {...}, "other": {...}}
```

SETTINGS THAT MATTER

SETTING	VALUE	WHY
SESSION_ENGINE	…backends.cache	Flow state lands in Redis and expires itself, rather than writing a `django_session` row per anonymous hand-off
SESSION_COOKIE_NAME	`surmount_session`	Cookies ignore the port — a partner's Django app on the same host would otherwise overwrite ours
SESSION_COOKIE_SAMESITE	Lax	The callback is a cross-site **top-level GET navigation**. `Lax` allows it; `Strict` would silently break every hand-off
SESSION_COOKIE_SECURE	`not DEBUG`	

CONFIGURATION MISTAKES THAT BREAK IT

SYMPTOM	CAUSE
`?error=sso_unavailable`	Unknown partner slug, missing `ORG_ID`, or a non-https issuer outside DEBUG
`?error=flow_expired` every time	Cookie not surviving the round trip — check `SESSION_COOKIE_NAME` collisions and `SameSite`
`?error=invalid_code` every time	`REDIRECT_URI` differs from what the partner registered — usually a trailing slash
Members land in the wrong tenant	`SSO_PARTNER_ORG_ID` points at the parent Surmount org. `Organization.clean()` forbids the skip-verification flags there

09 Error handling

SLUG	RAISED WHEN	MEMBER SEES	RETRY
`flow_expired`	No session, expired flow, `state` mismatch, replayed callback	“That took too long”	yes
`invalid_code`	Code spent, provider refused, ID token failed verification, provider unreachable	“We couldn't sign you in”	yes
`account_closed`	The resolved Surmount account is `is_deleted`	“Account closed — contact support”	no
`sso_unavailable`	Unknown partner, missing org — **our** misconfiguration	“Investing is unavailable”	no
`sso_failed`	Anything unhandled	Generic	yes

EXCEPTION MAPPING IN THE CALLBACK VIEW

EXCEPTION	FROM	→ SLUG
`MismatchingStateError`	authlib	`flow_expired`
`OAuthError`	authlib — provider refused, code would not exchange	`invalid_code`
`JoseError`	authlib — **ID token failed verification**	`invalid_code`
`SSOAccountClosed`	linking.py	`account_closed`
`SSOConfigurationError`	providers.py	`sso_unavailable`
`Exception`	anything else — logged with a trace	`sso_failed`

> **JOSEERROR IS NOT "UNHANDLED"**
>
> authlib raises `JoseError` — not `OAuthError` — for a forged, expired or replayed ID token. Catching it separately is what stops the refusal being logged as an unhandled crash. This was a real bug the tests caught; do not remove that branch.

TWO RULES

- **Slugs are coarse on purpose.** A callback that reported *why* it failed would be an oracle — an attacker could tell "already used" from "never existed". Five slugs from a closed set go in the URL; the real reason goes to the `sso.audit` log.
- **Never redirect to a sign-in page.** These accounts have a random password nobody knows. Every dead end returns the member to `PARTNER_HOME_URL`.

ERROR_DESCRIPTION IS DELIBERATELY NOT REFLECTED

Blossom reports refusals as `?error=…&error_description=…`. The description is attacker-influenceable text arriving over the front channel; reflecting it would turn our error page into a content mirror under our own domain. It goes to the audit log.

10 Edge cases

SITUATION	WHAT HAPPENS
Member changes their email at Blossom	Nothing breaks — we match on `sub`. The new address is picked up next hand-off
Two people, same email, different Blossom accounts	Two `sub` values → two links. The second hits the unique constraint on `(email, organization)` and fails loudly rather than merging two humans
Member deleted at Blossom	They can no longer sign in there, so they never reach us. Their Surmount account is untouched
Member suspended between the two legs	The provider re-checks at redemption rather than trusting the snapshot. No token is issued
Blossom rotates its signing key	Unknown `kid` → authlib refetches the JWKS once and continues. No downtime
Blossom rotates the client secret	Coordinated — both sides at once. Which is why the secret should be the less-rotated credential
Two tabs, two hand-offs at once	Works. authlib keys state per-`state` in the session, so concurrent flows coexist
Member's clock is wrong	Irrelevant. Every expiry check happens on a server
Server clocks disagree	60 s leeway on `exp` / `iat` — enough for NTP drift, not enough to extend an expired token
`picture` we will not render	Dropped; sign-in continues. Only `https` (plus `http` on localhost). A `javascript:` URL in an `<img src>` is script execution on our origin
Minimal ID token	We fetch the rest from `/userinfo` and check its `sub` matches. If they disagree, we take neither
`/userinfo` is down	Sign-in still succeeds. It only fills in decoration — a missing avatar must not block investing
Member blocks cookies	Refused with `flow_expired`. Correct — without the cookie there is no CSRF defence. But see §12, item 8
`?next=` with an absolute URL	Dropped. Only a path on our own app is accepted, and it is stored server-side, never round-tripped
A second partner arrives	An entry in `SSO_PARTNERS`. No route, no view, no branch

NOT YET DECIDED

- **Blossom changes a member's `sub`** — migration, IdP swap, tenant merge. The spec forbids it; providers do it anyway. There is no remediation path today, and the failure mode is a member permanently losing access to their brokerage account.
- **Two Blossom accounts, one human** (personal + business) → two `sub`s → two Surmount accounts. Probably fine, but undecided.
- **Member deletes their Surmount account and returns via SSO** — recreate, refuse, or restore? Currently refuses with `account_closed`.

11 Testing

RUNNING THE SUITES

```
# Surmount, the relying party — 48 tests
cd backend && .venv/bin/python -m pytest tests/sso/ -q

# Blossom, the reference provider — 33 tests
cd blossom-partner/backend && .venv/bin/python manage.py test oidc
```

HOW THE CLIENT TESTS WORK

- Everything is driven **through real HTTP**, via the Django test client, so what is proven is what ships.
- `conftest.py` holds a `FakeProvider` that owns a real RSA key and signs real JWTs — plus a **rogue key** the relying party has never seen, for forging.
- The provider is mocked with `responses`; discovery and JWKS are always answered, token and userinfo per test.
- `begin()` runs `/start/` and reads `state`, `nonce` and the verifier back out of the cache — the same way the callback does, because they never reach the browser.

WHAT IS COVERED

GROUP	CASES
Routing	Partner-scoped URLs, unknown partner, plain-http issuer refused outside DEBUG
Start	state/nonce/PKCE present, verifier absent from the URL, relative `next` stored, absolute `next` dropped
Happy path	Full hand-off, PKCE verifier presented, secret in the header not the body, `next` survives
Under attack	No flow, tampered state (and no network call made), replayed callback, unknown signing key, wrong `iss`, wrong `aud`, replayed nonce, expired token, `alg: none`, `/userinfo` `sub` mismatch
Failures	Provider refusal, token-endpoint failure, `/userinfo` down (non-fatal), error URL carries a slug and nothing else
Linking	Create, adopt, returning member, changed email, closed account (both lookups), picture validation
Session	Single-use redemption, unknown code, missing code

GAPS IN THE TEST SUITE

- ✕ **No concurrency tests.** The single-use claims depend on atomicity, and sequential tests pass on broken implementations. See §12, item 2.
- ✕ **No load or abuse testing.**
- ✕ **No test that two subjects cannot link to one user** — because they currently can. See §12, item 1.

12 Known gaps

Triaged from an external design review, checked against the code. Ordered by consequence. Several review findings did **not** apply and are listed at the end.

Fix now — small and real

1 · ONE SURMOUNT ACCOUNT CAN BE LINKED TO TWO BLOSSOM SUBJECTS

The adopt branch does not check whether the candidate is already linked:

- `linked_user(blossom, sub_B)` → None (a new subject)
- `find_existing_org_user(org, email)` → finds user **U**, already linked to `sub_A`
- `link_identity(blossom, sub_B, U)` → a **second** row pointing at the same user

`UNIQUE (partner_slug, subject)` does not prevent this. Combined with a partner whose email change does not re-verify, it is an account-takeover path.

Fix: refuse adoption if the candidate already carries a link for this partner. ~10 lines. **Effort:** hours.

2 · SINGLE-USE REDEMPTION IS NOT ATOMIC

`session.py` does `cache.get()` then `cache.delete()`. Two concurrent redemptions can both pass. Fix by checking the return value of `cache.delete()` — only one caller gets `True`. Add a **concurrency** test, not a sequential one. **Effort:** hours.

3 · DISCOVERY ISSUER IS NO LONGER VERIFIED — A REGRESSION

The previous hand-rolled adapter asserted the discovery document advertised the issuer we asked for. authlib does not, and it derives the `iss` check *from the metadata*. A hijacked discovery response could move both the token endpoint and the expected issuer. TLS is the real defence, but assert that `token_endpoint`, `jwks_uri` and `authorization_endpoint` share the issuer's origin. ~5 lines. **Effort:** hours.

4 · `_SAFE_NEXT` MISSES A BACKSLASH

We reject `//`, but `/\evil.com` passes and browsers normalise `\` to `/`. One line. **Effort:** minutes.

Before the partner integration call

#	ITEM	EFFORT
5	`Referrer-Policy: no-referrer` and `history.replaceState` on the Whitelabel callback route; drop `session_code` to 30 s. Any third-party asset on that route leaks the handle in a `Referer` header within milliseconds	half day
6	JWKS refetch cooldown — one per issuer per 60 s. Today an attacker can force one fetch per distinct unknown `kid`, making us a DoS amplifier pointed at the partner	half day
7	Circuit breaker on Blossom calls. Ten seconds × concurrent sign-ins during an outage is worker-pool exhaustion, which takes down more than SSO	1 day
8	Cookie-blocked members get `flow_expired`, which offers a retry that can never work. Probe for the cookie at `/start/` and return a distinct non-retryable slug	half day
9	Per-slug failure-rate dashboard. It is how you learn Blossom rotated a key or broke `email_verified` before members tell you	1 day

Real, but a project — needs a decision

10 · REVOCATION DOES NOT PROPAGATE

- If Blossom suspends a member tomorrow, they stay signed into Surmount for **up to 30 days**. Nothing tells us.
- For a bank→broker link this is a contractual issue, not a backlog item, and it will surface in Blossom's vendor security review.
- **Fix:** OIDC Back-Channel Logout, or — as a day-one stopgap — a client-authenticated `POST /api/sso/blossom/revoke/` giving their fraud team a lever.

Effort: 2–4 days for the stopgap; the full fix shares infrastructure with item 11.

11 · A 60-DAY BEARER TOKEN IN JS-READABLE STORAGE

- `ACCESS_TOKEN_LIFETIME_MINUTES` defaults to **86400 (60 days)**; local sets ~2 years. Refresh is 30 days.
- The Whitelabel stores the pair in `localStorage` (AES there is obfuscation, not protection — the key ships in the bundle).
- We took great care that a JWT never reaches a **URL**, where reading it requires reading a log. It sits in storage, where reading it requires one line of injected script — on a Blossom-branded page that will accumulate marketing and analytics tags.

Cheapest large win: cut the access token to ≤15 minutes with refresh rotation and reuse detection. **Correct end state:** an `HttpOnly` first-party cookie session, which requires serving Surmount under `invest.blossom.com/api` — and which makes item 10 nearly free, since revocation becomes a `DELETE`.

Effort: hours for the lifetime cut; ~1 week plus infrastructure for the cookie session.

#	ITEM	EFFORT
12	**Mobile scope.** If the link ships in Blossom's app, embedded webviews may not share cookies with the system browser and the flow fails 100% of the time. RFC 8252 prescribes `ASWebAuthenticationSession` / Chrome Custom Tabs. Decide, and either build it or state the omission	hours to decide
13	**Audit log specification.** `sso.audit` is used as a security control but its schema, retention and PII handling are unspecified. For a broker-dealer, sign-in provenance is a regulated record	1 day
14	**Pre-KYC account state.** An SSO-created account exists before KYC has run and must not be able to trade or move money. Today that is an implicit consequence rather than an enforced state	decide
15	**Unlink.** No path for "disconnect Investments from my Blossom account"	2 days

Considered and deferred

- **PAR (RFC 9126)** — moves `state`, `nonce` and the challenge off the front channel entirely. Cheap if Blossom's provider is greenfield; do not block the integration on it.
- **`private_key_jwt` client authentication** — replaces the symmetric client secret with a key pair, making secret rotation a non-event. Genuinely the more consistent choice given how much we argue for asymmetry elsewhere.
- **`max_age` / `auth_time` freshness** on the create and adopt branches — adds a redirect and a password prompt to a flow whose entire promise is "one tap". Revisit if Blossom's sessions turn out to be long.
- **Consent and data-sharing agreement** — not code, but it can block launch. Owned by whoever holds the Blossom contract.

Review findings that did not apply

FINDING	REALITY
`x-organization-id` is a tenancy bypass	`OrganizationMiddleware` **403s** (`org_mismatch_rejected`) when an authenticated user's org differs from the header. On `/session/` the org comes from the code, not the header
No rate limiting	`SSOThrottle`, scope `sso`, 30/min on all three endpoints
`?next=` round-trips through the front channel	Stored server-side in the session; `//` already rejected
Two tabs break each other	Fixed by the authlib rewrite — state is keyed per-`state`, so flows coexist
The token-exchange fallback adapter will rot	Already deleted. The seam remains, so a future non-OIDC partner is a file, not a refactor

13 Adding a second partner

1 Get their three values

Issuer, client ID, client secret. Send them the Blossom Developer Guide — it is partner-agnostic apart from the URLs.

2 Create their Surmount organization

Not the parent org. `Organization.clean()` forbids the skip-verification flags there, and those flags are what let a hand-off finish without an email round trip.

3 Add an entry to `SSO_PARTNERS`

In production, set `SSO_PARTNERS_JSON` to the whole map — then adding a partner is a secret rotation rather than a release.

4 Give them their callback URL

`https://api.surmount.ai/api/sso/<their-slug>/callback/` — the slug is the map key.

5 Deploy a whitelabel for them

`NEXT_PUBLIC_PARTNER_SLUG`, `NEXT_PUBLIC_PARTNER_NAME`, `NEXT_PUBLIC_ORG_ID`, `NEXT_PUBLIC_PARTNER_HOME_URL`. Add their avatar host to `next.config.ts` `remotePatterns`.

No route, no view, no branch. If any of those turn out to be needed, the seam has been violated somewhere and that is the bug to fix.

A PARTNER WITH NO OPENID PROVIDER

We removed the token-exchange fallback to keep one path. If a partner genuinely cannot run a provider, write a second adapter behind `Identity` — a file, not a refactor. But push hard on the provider first: every mainstream stack has a library, and the alternative is a symmetric secret that lets both sides mint what the other verifies.

14 FAQ

Why do we mint our own session instead of relaying Blossom's token?

Token relay would remove the revocation problem entirely — Blossom's logout would be instantly effective. But it couples us to their availability on **every API call**, not just at sign-in, so a Blossom outage becomes a Surmount outage. Every future partner would need bespoke token validation, killing the seam. And our direct, non-partner users need their own auth path anyway, so we would maintain two session models instead of one. For a multi-partner platform this is the right call; if Blossom were the only partner forever, relay would be the simpler system.

Why is `/start/` not in any spec?

Because OIDC only specifies the provider's endpoints. A relying party only has to have a `redirect_uri`, and even that path is ours to name. Putting the provider in the path is the mainstream convention — Spring Security, django-allauth and NextAuth all do it.

Why does the button on Blossom's page point at us?

The flow must begin wherever `state`, `nonce` and the PKCE verifier are generated and stored — the relying party. If Blossom started it they would have to generate our state and know our client config. It also means the same URL serves a "Continue with Blossom" button on our own app.

Why does the Whitelabel not decode the JWT?

A token is a claim, and only the backend can verify it. If the frontend trusted its contents, editing localStorage would let anyone claim to be anyone. It calls `/api/user/detail/`.

Why route account creation through `SignupAPIView`?

That is where the per-org flags actually take effect — `skip_email_verification`, cross-tenant KYC linkage, banners, activity tracking. Calling `create_user` directly would skip all of it and leave partner accounts subtly different from every other account in ways that surface months later.

Why is the flow window an hour when everything else is 60 seconds?

It has to survive a member typing a password and passing an MFA prompt on Blossom's login page. It is authlib's default and it is single-use, so the exposure is a stale record rather than a live credential.

What is the blast radius if the client secret leaks?

An attacker who **also** obtains an authorization code within its 60-second life, **and** the PKCE verifier, could redeem it. The verifier never leaves our server, so in practice the secret alone is not enough. Rotate it, coordinated with Blossom.

What if we need to support a partner on a different protocol — SAML, or something bespoke?

Write an adapter that returns an `Identity`. Nothing below the seam changes. That is the whole reason the seam exists.

Partner SSO · Surmount Developer Guide · OpenID Connect, Authorization Code with PKCE. Every value in this document is taken from the running implementation on `feat/blossom-partner-sso`. §12 is the part to read before shipping.